1a AIM: To implement a Simple Linear Regression algorithm using Python for salary prediction and evaluate its performance using Mean Squared Error (MSE), Mean Absolute Error (MAE), and R^2 Score.

ALGORITHM:

1. Import the required libraries.
2. Load the dataset containing years of experience and corresponding salary values.
3. Separate the independent variable (Years of Experience) and dependent variable (Salary).
4. Divide the dataset into training and testing sets.
5. Calculate the mean values of the training data for both experience and salary.
6. Compute the slope (m) of the regression line using the formula:

$$m = \frac{\sum(x_i - x)(y_i - y)}{\sum(x_i - x)^2}$$

7. Calculate the intercept (c) using the formula:

$$c = y - mx$$

8. Form the regression equation: $y = mx + c$
   where y represents the predicted salary and x represents the years of experience.
9. Predict the salary values for the test data using the regression equation.
10. Calculate the Mean Absolute Error (MAE) using

$$MAE = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|$$

11. Calculate the Mean Squared Error (MSE) using

$$MSE = \frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

12. Calculate the Root Mean Squared Error (RMSE) using

$$RMSE = \sqrt{MSE}$$

13. Calculate the Coefficient of Determination (R^2 Score) using

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - y)^2}$$

14. Predict the salary for a new employee based on the given years of experience.
15. Display the regression equation and performance metrics.
16. Analyze the relationship between experience and salary.
17. End.

PROGRAM:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
df = pd.read_csv("LinearRegression_Exp1.csv")
X = df['YearsExperience'].values
```

```
Y = df['Salary'].values
X_train, X_test, Y_train, Y_test = train_test_split(
   X, Y, test_size=0.2, random_state=42)
x_mean = np.mean(X_train)
y_mean = np.mean(Y_train)
numerator = np.sum((X_train - x_mean) * (Y_train - y_mean))
denominator = np.sum((X_train - x_mean) ** 2)
m = numerator / denominator
c = y_mean - m * x_mean
print("Slope (m) =", m)
print("Intercept (c) =", c)
Y_pred = m * X_test + c
mae = np.sum(np.abs(Y_test - Y_pred)) / len(Y_test)
mse = np.sum((Y_test - Y_pred) ** 2) / len(Y_test)
rmse = np.sqrt(mse)
ss_total = np.sum((Y_test - np.mean(Y_test)) ** 2)
ss_residual = np.sum((Y_test - Y_pred) ** 2)
r2 = 1 - (ss_residual / ss_total)
print("\nMean Absolute Error =", mae)
print("Mean Squared Error =", mse)
print("Root Mean Squared Error =", rmse)
print("R2 Score =", r2)
experience = 6.5
predicted_salary = m * experience + c
print("\nPredicted Salary for", experience, "years of experience =", predicted_salary)
```

OUTPUT

```
Slope (m) = 9449.962321455077
Intercept (c) = 24852.72834264744
Mean Absolute Error = 6286.453830757749
Mean Squared Error = 49830096.85590839
Root Mean Squared Error = 7059.043621048624
R2 Score = 0.9024461774180497
Predicted Salary for 6.5 years of experience = 86377.48343159044
```

1b AIM: To implement a Multiple Linear Regression Algorithm using Python and evaluate its performance using Mean Squared Error (MSE), Mean Absolute Error (MAE), and R^2 Score.

ALGORITHM:

1. Import the required libraries.
2. Load the insurance dataset.
3. Convert the categorical attributes such as gender and smoking habits into numerical values.
4. Select age, BMI, number of children, smoking habits, and gender as input variables and insurance charges as the output variable.
5. Divide the dataset into training and testing sets.
6. Add a column of ones to the training data to represent the intercept.
7. Calculate the regression coefficients using
$$f3 = (X^T X)^{-1} X^T Y$$
8. Obtain the intercept and coefficients from the parameter vector.
9. Predict insurance charges for the test data using
$$\hat{Y} = Xf3$$
10. Calculate Mean Absolute Error (MAE) using
$$MAE = \frac{1}{n} \sum_{i=1}^{n} |y_i - y_i|$$
11. Calculate Mean Squared Error (MSE) using
$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - y_i)^2$$
12. Calculate Root Mean Squared Error (RMSE) using
$$RMSE = \sqrt{MSE}$$
13. Calculate the Coefficient of Determination (R^2 Score) using
$$R^2 = 1 - \frac{\sum (y_i - y_i)^2}{\sum (y_i - \bar{y})^2}$$
14. Display the regression equation, coefficients, and performance metrics.
15. Analyze the effect of each factor on insurance charges.
16. End.

PROGRAM:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
df = pd.read_csv("MultipleLinearRegression_Exp2.csv")
df['sex'] = df['sex'].map({'male':1, 'female':0})
df['smoker'] = df['smoker'].map({'yes':1, 'no':0})
X = df[['age', 'bmi', 'children', 'smoker', 'sex']].values
Y = df['charges'].values
X_train, X_test, Y_train, Y_test = train_test_split(
   X, Y, test_size=0.2, random_state=42)
ones = np.ones((X_train.shape[0],1))
X_train = np.concatenate((ones, X_train), axis=1)
beta = np.linalg.inv(X_train.T.dot(X_train)).dot(X_train.T).dot(Y_train)
```

```
intercept = beta[0]
coefficients = beta[1:]
print("Intercept =",
intercept)
print("\nCoefficients:")
print("Age =",
coefficients[0]) print("BMI
=", coefficients[1])
print("Children =",
coefficients[2]) print("Smoker
=", coefficients[3]) print("Sex =",
coefficients[4])
ones_test = np.ones((X_test.shape[0],1))
X_test = np.concatenate((ones_test, X_test),
axis=1) Y_pred = X_test.dot(beta)
mae = np.sum(np.abs(Y_test - Y_pred)) /
len(Y_test) mse = np.sum((Y_test - Y_pred)**2) /
len(Y_test) rmse = np.sqrt(mse)
ss_total = np.sum((Y_test -
np.mean(Y_test))**2) ss_residual =
np.sum((Y_test - Y_pred)**2)
r2 = 1 - (ss_residual / ss_total)
print("\nMean Absolute Error =", mae)
print("Mean Squared Error =", mse)
print("Root Mean Squared Error =",
rmse) print("R2 Score =", r2)
new_customer = np.array([1, 35, 28.5, 2, 1, 1])
predicted_charge = new_customer.dot(beta)
print("\nPredicted Insurance Charge =",
predicted_charge)
```

SAMPLE OUTPUT

```
Intercept = -
11938.65
Coefficients:
Age = 256.84
BMI = 339.29
Children = 475.50
Smoker = 23848.53
Sex = -131.36
Mean Absolute Error = 4180.42
Mean Squared Error =
35500000.74 Root Mean Squared
Error = 5958.19 R2 Score = 0.87
Predicted Insurance Charge = 14562.78
```

2:a To implement the k-Nearest Neighbors algorithm and evaluate its performance using accuracy, confusion matrix, precision, recall, and F1-score.
ALGORITHM:

1. Import the required libraries.
2. Load the customer purchase dataset.
3. Separate input features and target labels.
4. Split the dataset into training and testing sets.
5. Initialize K and calculate Euclidean distances.
6. Sort distances and select the K nearest neighbors.
7. Predict the class using majority voting.
8. Repeat prediction for all test samples.
9. Calculate accuracy, precision, recall, and F1-score.
10. Generate the confusion matrix.
11. Analyze different values of K.
12. Display the results and end.

PROGRAM:

```
import pandas as pd
import numpy as np
from collections import Counter
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
df = pd.read_csv("KNN_Exp3.csv")
X = df[['AnnualIncome', 'PurchaseFrequency']].values
y = df['SpendingCategory'].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
k = 3
def euclidean_distance(x1, x2):
    distance = np.sqrt(np.sum((x1 - x2) ** 2))
    return distance
def predict(X_train, y_train, test_point, k):
    distances = []
    for i in range(len(X_train)):
        distance = euclidean_distance(test_point, X_train[i])
        distances.append((distance, y_train[i]))
    distances.sort(key=lambda x: x[0])
    neighbors = distances[:k]
    labels = [neighbor[1] for neighbor in neighbors]
    predicted_class = Counter(labels).most_common(1)[0][0]
    return predicted_class
y_pred = []
for test_point in X_test:
    prediction = predict(X_train, y_train, test_point, k)
    y_pred.append(prediction)
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy =", accuracy)
cm = confusion_matrix(y_test, y_pred)



print("\nConfusion Matrix:")
print(cm)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

SAMPLE OUTPUT

```
Accuracy = 0.95

Confusion Matrix:
[[12  0  1]
 [ 0 10  1]
 [ 0  0 16]]

Classification Report:

              precision    recall  f1-score   support

        High       1.00      0.92      0.96        13
         Low       1.00      0.91      0.95        11
      Medium       0.89      1.00      0.94        16

    accuracy                           0.95        40
   macro avg       0.96      0.94      0.95        40
weighted avg       0.95      0.95      0.95        40
```

2:b To implement the k-NN Regression algorithm and evaluate its performance using MSE, MAE, and R^2 Score.

ALGORITHM:

1. Import the required libraries.
2. Load the house price dataset.
3. Convert categorical variables into numerical values.
4. Separate input features and target values.
5. Split the dataset into training and testing sets.
6. Initialize K and calculate Euclidean distances.
7. Sort distances and select the K nearest neighbors.
8. Predict the price using the mean of neighbor values.
9. Repeat prediction for all test samples.
10. Calculate MAE, MSE, RMSE, and R^2 score.
11. Analyze different values of K.
12. Display the results and end.

PROGRAM:

```
import pandas as pd
import numpy as np
from collections import Counter
from sklearn.model_selection import train_test_split
df = pd.read_csv("KNNRegression_Exp4.csv")
location_mapping = {'Urban':0, 'Suburban':1, 'Rural':2}
df['Location'] = df['Location'].map(location_mapping)
X = df[['Area', 'Location', 'Bedrooms']].values
y = df['Price'].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
k = 3
def euclidean_distance(x1, x2):
    return np.sqrt(np.sum((x1 - x2)**2))
def predict(X_train, y_train, test_point, k):
    distances = []
    for i in range(len(X_train)):
        distance = euclidean_distance(test_point, X_train[i])
        distances.append((distance, y_train[i]))
    distances.sort(key=lambda x: x[0])
    neighbors = distances[:k]
    neighbor_prices = [neighbor[1] for neighbor in neighbors]
    predicted_price = np.mean(neighbor_prices)
    return predicted_price
y_pred = []
for test_point in X_test:
    prediction = predict(X_train, y_train, test_point, k)
    y_pred.append(prediction)
y_pred = np.array(y_pred)
mae = np.sum(np.abs(y_test - y_pred)) / len(y_test)
mse = np.sum((y_test - y_pred)**2) / len(y_test)
```

```
rmse = np.sqrt(mse)
ss_total = np.sum((y_test - np.mean(y_test))**2)



ss_residual = np.sum((y_test - y_pred)**2)
r2 = 1 - (ss_residual / ss_total)
print("Mean Absolute Error =", mae)
print("Mean Squared Error =", mse)
print("Root Mean Squared Error =", rmse)
print("R2 Score =", r2)
for k in [1, 3, 5, 7, 9]:
    y_pred = []
    for test_point in X_test:
        prediction = predict(X_train, y_train, test_point, k)
        y_pred.append(prediction)
    y_pred = np.array(y_pred)
    mse = np.sum((y_test - y_pred)**2) / len(y_test)
    rmse = np.sqrt(mse)
    print("K =", k, " RMSE =", rmse)
```

Sample Output

K = 1 RMSE = 52000.35
K = 3 RMSE = 41500.92
K = 5 RMSE = 37200.84
K = 7 RMSE = 40150.77
K = 9 RMSE = 43800.21

From the above observations, **K = 5** produces the minimum RMSE and hence gives the best prediction accuracy.

Mean Absolute Error = 28500.42

Mean Squared Error = 1383902500.68

Root Mean Squared Error = 37200.84

R2 Score = 0.91

EXP3 Implement a Decision Tree algorithm to classify loan applications and understand the decision-making process.

AIM: To implement the Decision Tree algorithm and evaluate its performance using accuracy, confusion matrix, precision, recall, and F1-score.

ALGORITHM: 1. Import the required libraries. 2. Load the loan approval dataset. 3. Convert categorical variables such as employment status and credit history into numerical values. 4. Separate the input attributes and target labels. 5. Divide the dataset into training and testing sets. 6. Calculate the entropy of the target attribute 7. Calculate the Information Gain for each attribute 8. Select the attribute with the highest Information Gain as the splitting attribute. 9. Repeat the process recursively until all leaf nodes become pure. 10. Construct the Decision Tree. 11. Predict the class labels for the test samples. 12. Calculate classification accuracy 13. Construct the confusion matrix. 14. Calculate Precision 15. Calculate 16. Calculate F1-Score 17. Display the classification results. 18. Analyze the decision-making process. 19. End.

PROGRAM

```
import pandas as pd
import numpy as np
from collections import Counter
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
df = pd.read_csv("DecisionTree_Exp5.csv")
df['EmploymentStatus'] = df['EmploymentStatus'].map(
   {'Unemployed': 0, 'Self-employed': 1, 'Employed': 2})
df['CreditHistory'] = df['CreditHistory'].map(
   {'Bad': 0, 'Average': 1, 'Good': 2})
df['LoanStatus'] = df['LoanStatus'].map(
   {'Rejected': 0, 'Approved': 1})
X = df[['Income', 'EmploymentStatus', 'CreditHistory']].values
y = df['LoanStatus'].values
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42)
def entropy(y):
   counts = Counter(y)
   probabilities = [count / len(y) for count in counts.values()]
   return -sum(p * np.log2(p) for p in probabilities if p > 0)
def information_gain(X_column, y):
   parent_entropy = entropy(y)
   values = np.unique(X_column)
   weighted_entropy = 0
   for value in values:
      subset_indices = np.where(X_column == value)[0]
      subset_y = y[subset_indices]
      weighted_entropy += (
         len(subset_y) / len(y)
      ) * entropy(subset_y)
   return parent_entropy - weighted_entropy
```

```
best_gain = -1
best_feature = -1
for i in range(X_train.shape[1]):
    gain = information_gain(X_train[:, i], y_train)
    if gain > best_gain:
        best_gain = gain
        best_feature = i
feature_names = ['Income', 'EmploymentStatus', 'CreditHistory']
print("Best Splitting Attribute =", feature_names[best_feature])
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(criterion='entropy')
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy =", accuracy)
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred,
    target_names=['Rejected', 'Approved']))
```

OUTPUT

Best Splitting Attribute = CreditHistory

Accuracy = 0.8

Confusion Matrix:
[[2 1]
 [0 2]]

Classification Report:

	precision	recall	f1-score	support
Rejected	1.00	0.67	0.80	3
Approved	0.67	1.00	0.80	2
accuracy			0.80	5
macro avg	0.83	0.83	0.80	5
weighted avg	0.87	0.80	0.80	5

4a AIM:To develop an SVM classifier and evaluate its performance in disease prediction.

ALGORITHM:

1. Import the required libraries.
2. Load the breast cancer dataset.
3. Separate the input attributes and target labels.
4. Divide the dataset into training and testing sets.
5. Standardize the feature values.
6. Initialize the Support Vector Machine classifier.
7. Determine the optimal hyperplane
8. Identify the support vectors that are closest to the hyperplane.
9. Classify the test samples based on the sign of the decision function.
10. Predict the tumor classes for the test data.
11. Calculate classification accuracy using
12. Construct the confusion matrix.
13. Calculate Precision
14. Calculate Recall
15. Calculate F1-Score
16. Display the classification results.
17. Analyze the performance of the SVM model.
18. End.

PROGRAM:

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.svm import SVC
data = load_breast_cancer()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = SVC(kernel='linear')
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy =", accuracy)
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)
print("\nClassification Report:")
print(classification_report(
```

```
    y_test,
    y_pred,
  target_names=['Malignant', 'Benign']))
```

SAMPLE OUTPUT

```
Accuracy = 0.9824561403508771

Confusion Matrix:
[[41  2]
 [ 0 71]]

Classification Report:
              precision    recall  f1-score   support

   Malignant       1.00      0.95      0.98        43
      Benign       0.97      1.00      0.99        71

    accuracy                           0.98       114
   macro avg       0.99      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114
```

4b AIM:To Develop an SVR Algorithm to predict future sales and compare its performance with traditional regression techniques.

ALGORITHM:

1. Import the required libraries.
2. Load the sales dataset.
3. Separate the input variables and output variable.
4. Divide the dataset into training and testing sets.
5. Standardize the feature values.
6. Initialize the Support Vector Regression model with an RBF kernel.
7. Train the model using the training data.
8. Predict sales values for the test data.
9. Calculate Mean Absolute Error (MAE)
10. Calculate Mean Squared Error (MSE)
11. Calculate Root Mean Squared Error (RMSE) using
12. Calculate the Coefficient of Determination (R^2 Score) using
13. Display the regression metrics.
14. Analyze the prediction performance.
15. End.

PROGRAM:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR
df = pd.read_csv("SVR_Exp7.csv")
X = df[['Advertising', 'SeasonalDemand', 'Promotions']].values
y = df['Sales'].values

X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = SVR(kernel='rbf')
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
mae = np.sum(np.abs(y_test - y_pred)) / len(y_test)
mse = np.sum((y_test - y_pred) ** 2) / len(y_test)
rmse = np.sqrt(mse)
ss_total = np.sum((y_test - np.mean(y_test)) ** 2)
ss_residual = np.sum((y_test - y_pred) ** 2)
r2 = 1 - (ss_residual / ss_total)
print("Mean Absolute Error =", mae)
print("Mean Squared Error =", mse)
print("Root Mean Squared Error =", rmse)
print("R2 Score =", r2)
```

SAMPLE OUTPUT

Mean Absolute Error = 1264.78

Mean Squared Error = 2846537.92

Root Mean Squared Error = 1687.76

R2 Score = 0.93

5: K-Means Clustering Algorithm

AIM

To implement the K-Means clustering algorithm to segment customers based on their purchasing behavior and identify distinct customer groups for designing targeted marketing campaigns.

ALGORITHM

1. Import the required libraries.
2. Load the customer dataset.
3. Select the features representing purchasing behavior.
4. Initialize the number of clusters K.
5. Create a KMeans object from the sklearn.cluster module, specifying the number of clusters.
6. Fit the KMeans model to the input data using the fit() method.
7. Obtain the cluster label assigned to each data point using the labels_ attribute.
8. Obtain the final cluster centroids using the cluster_centers_ attribute.
9. Display the final cluster labels and centroids.
10. Analyze the customer groups obtained.
11. End.

PROGRAM

```
import pandas as pdfrom sklearn.cluster import KMeansdf =
pd.read_csv("KMeans_Exp8.csv")X = df[['AnnualIncome', 'SpendingScore']].valuesK =
3kmeans = KMeans(n_clusters=K, init='k-means++', n_init=10,
random_state=42)kmeans.fit(X)labels = kmeans.labels_centroids =
kmeans.cluster_centers_print("Final Centroids:\n")print(centroids)for k in range(K):
print("\nCluster", k)    print("Centroid =", centroids[k])    print(X[labels ==
k])print("\nCluster 0 : Low Income - Low Spending Customers")print("Cluster 1 : High
Income - High Spending Customers")print("Cluster 2 : Moderate Income - Moderate
Spending Customers")
```

SAMPLE OUTPUT

```
Final Centroids:

[[25.63 20.84]
 [86.27 82.43]
 [54.18 49.72]]

Cluster 0
Centroid = [25.63 20.84]
[[15 39]
 [16 81]
 [17  6]
 [18 77]
 [19 40]]

Cluster 1
Centroid = [86.27 82.43]
[[70 95]
 [71 85]
 [73 88]
 [75 90]
```

[78 97]]

Cluster 2
Centroid = [54.18 49.72]
[[40 55]
[42 48]
[45 50]
[48 52]
[50 60]]

Cluster 0 : Low Income - Low Spending Customers
Cluster 1 : High Income - High Spending Customers
Cluster 2 : Moderate Income - Moderate Spending Customers
Note: Keep KMeans_Exp8.csv in the same folder as your Python program. The output shown is the sample output from the lab manual; actual results depend on the dataset.

6: Principal Component Analysis (PCA)

Aim

To reduce gene expression features using PCA, plot the first two principal components, and calculate their explained variance.

Algorithm

1. Import NumPy, Pandas, Matplotlib, and StandardScaler.
2. Read the gene expression CSV file and standardize all numerical features.
3. Calculate the feature means and center the standardized data.
4. Form the covariance matrix and calculate eigenvalues and eigenvectors.
5. Sort eigenvalues in descending order and choose the first two eigenvectors.
6. Project the centered data onto these principal components.
7. Calculate explained variance ratios, display them, and plot the two-dimensional data.

Program

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("PCA_Exp9.csv")
X = StandardScaler().fit_transform(df.values)
X = X - np.mean(X, axis=0)

cov = np.cov(X.T)
values, vectors = np.linalg.eigh(cov)
order = np.argsort(values)[::-1]
values, vectors = values[order], vectors[:, order]

Z = X @ vectors[:, :2]
ratio = values / values.sum()
print("PC1:", round(ratio[0], 4))
print("PC2:", round(ratio[1], 4))
print("Total variance retained:", round(ratio[:2].sum(), 4))

plt.scatter(Z[:, 0], Z[:, 1])
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA Projection")
plt.show()
```

Sample Output

PC1: 0.4427
PC2: 0.1897
Total variance retained: 0.6324
Scatter plot: Each sample is represented by a point, with PC1 on the X-axis and PC2 on the Y-axis.

Output Interpretation

• The covariance matrix summarizes relationships between the standardized features.

• The eigenvectors give principal component directions, and eigenvalues measure variance along them.

• PC1 retains 44.27% and PC2 retains 18.97% of the variance. Together they retain 63.24%.

• The two-dimensional scatter plot helps identify patterns, similarities, and possible clusters in the samples.

Result

Thus, PCA was implemented to reduce gene expression data to two principal components, visualize the reduced dimensions, and measure the proportion of variance retained. The sample output retains 63.24% of the total variance.

7 Feedforward Neural Network Using Backpropagation

Algorithm

1. Import libraries and load the loan default CSV dataset.
2. Separate input features and target labels; split data into training and test sets.
3. Standardize features using the training data.
4. Initialize input-to-hidden and hidden-to-output weights and biases.
5. Perform forward propagation using sigmoid activation in both layers.
6. Calculate prediction error and backpropagate it through the network.
7. Update weights and biases over multiple epochs.
8. Predict test labels using a threshold of 0.5 and print evaluation metrics.

Program

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

# Load and split dataset
df = pd.read_csv("NeuralNetwork_Exp10.csv")
X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values.reshape(-1, 1)
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42)

sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# Initialize weights and biases
np.random.seed(42)
W1 = np.random.randn(X_train.shape[1], 5)
b1 = np.zeros((1, 5))
W2 = np.random.randn(5, 1)
b2 = np.zeros((1, 1))

def sigmoid(x):
   return 1 / (1 + np.exp(-x))

def derivative(x):
   return x * (1 - x)

# Training using backpropagation
lr = 0.01
for epoch in range(1000):
   A1 = sigmoid(X_train @ W1 + b1)
   y_pred = sigmoid(A1 @ W2 + b2)

   error = y_train - y_pred
   d2 = error * derivative(y_pred)
   d1 = (d2 @ W2.T) * derivative(A1)
```

```
    W2 += lr * (A1.T @ d2)
    b2 += lr * np.sum(d2, axis=0, keepdims=True)
    W1 += lr * (X_train.T @ d1)
    b1 += lr * np.sum(d1, axis=0, keepdims=True)

# Testing
A1 = sigmoid(X_test @ W1 + b1)
output = sigmoid(A1 @ W2 + b2)
pred = (output >= 0.5).astype(int)

print("Accuracy =", accuracy_score(y_test, pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, pred))
print("Classification Report:\n",
    classification_report(y_test, pred))
```

Sample Output

```
Accuracy = 0.92

Confusion Matrix:
[[34  2]
 [ 4 40]]

Classification Report:

              precision    recall  f1-score   support

           0       0.89      0.94      0.92        36
           1       0.95      0.91      0.93        44

    accuracy                           0.92        80
   macro avg       0.92      0.92      0.92        80
weighted avg       0.92      0.92      0.92        80
```

Ex.8 AIM: develop a Convolutional Neural Network (CNN) model for classifying plant leaf images into different disease categories, understand convolution, pooling, and feature extraction operations, and evaluate the model using classification metrics.

ALGORITHM:

1. Import the required libraries.
2. Load and normalize the image dataset.
3. Apply convolution and ReLU activation.
4. Perform max pooling.
5. Flatten the feature map.
6. Calculate the hidden layer output and apply sigmoid activation.
7. Calculate output values and probabilities.
8. Compute the error.
9. Update weights and biases using backpropagation.
10. Predict class labels.
11. Calculate accuracy, precision, recall, and F1-score.
12. Display the results and end.

PROGRAM:

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
```



```
digits = load_digits()
X = digits.images
y = digits.target
y = (y >= 5).astype(int)
X = X / 16.0
X = X.reshape(-1, 8, 8, 1)
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42)
model = Sequential([
Conv2D(8, (3, 3), activation='relu', input_shape=(8, 8, 1)),
MaxPooling2D((2, 2)),
Flatten(),
Dense(16, activation='relu'),
Dense(1, activation='sigmoid')])
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=20, batch_size=8, verbose=0)
y_prob = model.predict(X_test)
y_pred_class = (y_prob >= 0.5).astype(int)
accuracy = accuracy_score(y_test, y_pred_class)
print("Accuracy =", accuracy)
cm = confusion_matrix(y_test, y_pred_class)
print("\nConfusion Matrix:")
print(cm)
# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred_class))
```

SAMPLE OUTPUT

Accuracy = 0.94

60

Confusion Matrix

[[166 11]

[9 174]]

Classification Report

precision recall f1-score

0 0.95 0.94 0.94

1 0.94 0.95 0.95

accuracy 0.94

macro avg 0.94 0.94 0.94

weighted avg 0.94 0.94 0.94

EX: 9 AIM : To implement a Recurrent Neural Network (RNN) for sentiment analysis of text data.

ALGORITHM:

1. Import the required libraries.
2. Load the customer review dataset and convert reviews to lowercase.
3. Build the vocabulary and assign word indices.
4. Convert reviews into sequences and pad them.
5. Split the dataset into training and testing sets.
6. Initialize RNN weights and biases.
7. Compute hidden states and output probabilities using sigmoid activation.
8. Calculate prediction error and update weights through training.
9. Repeat training for the specified epochs.
10. Predict sentiment classes for test reviews.
11. Generate the confusion matrix (TP, TN, FP, FN).
12. Calculate accuracy, precision, recall, and F1-score.
13. Display and analyze the classification results.

PROGRAM:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense
df = pd.read_csv("RNN_Exp12.csv")
reviews = df['Review'].str.lower().values
labels = df['Sentiment'].values
vocab_size = 500
max_length = 20
tokenizer = Tokenizer(num_words=vocab_size, oov_token="<OOV>")
tokenizer.fit_on_texts(reviews)
sequences = tokenizer.texts_to_sequences(reviews)
padded_sequences = pad_sequences(sequences, maxlen=max_length, padding='post')
X_train, X_test, y_train, y_test = train_test_split(
padded_sequences, labels, test_size=0.2, random_state=42)
model = Sequential([
Embedding(vocab_size, 8, input_length=max_length),
SimpleRNN(8, activation='tanh'),
Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=20, batch_size=8, verbose=0)
y_prob = model.predict(X_test)
y_pred_class = (y_prob >= 0.5).astype(int)
accuracy = accuracy_score(y_test, y_pred_class)
print("Accuracy =", accuracy)
cm = confusion_matrix(y_test, y_pred_class)
print("\nConfusion Matrix")
```

```
print(cm)
report = classification_report(y_test, y_pred_class, output_dict=True)
print("\nPrecision =", round(report['1']['precision'], 4))
print("Recall =", round(report['1']['recall'], 4))
print("F1 Score =", round(report['1']['f1-score'], 4))
```

SAMPLE OUTPUT

```
Accuracy = 0.91
Confusion Matrix
[[43 4]
[ 5 48]]
Precision = 0.9231
Recall = 0.9057
F1 Score = 0.9143
```